

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 13
Operating Systems
Topic: Semaphores in Your Own Kernel (Track B - xv6)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Laboratory Objectives 3

2 PART 1: KERNEL SEMAPHORE ARCHITECTURE **4**

2.1 System Architecture and Data Structures 4

2.2 System Call Registration 4

2.3 System Call Implementations (`kernel/sem.c`) 5

3 PART 2: TEST SUITE VERIFICATION (`semtest.c`) **7**

3.1 Test Program Logic 7

3.2 Execution Output 8

4 PART 3: MULTIPROCESSING INTERLEAVING WITH `mix.c` **9**

4.1 Console Contention Analysis 9

4.2 Execution Without Synchronization (`mix`) 9

4.3 Execution With Semaphore Locking (`mix lock`) 9

5 OBSERVATION QUESTIONS AND ARCHITECTURAL ANALYSIS **11**

6 CONCLUSIONS **13**

REFERENCES **14**

1 INTRODUCTION AND GOAL

1.1 Introduction

Synchronization inside an operating system kernel is fundamentally more challenging than user-space synchronization. Within the kernel, code executes with elevated hardware privileges (Supervisor mode on RISC-V), manages raw hardware devices, and coordinates multiple physical processors (SMP) while servicing asynchronous hardware interrupts.

Operating system kernels typically utilize two major classes of synchronization mechanisms:

- **Spinlocks:** Polling locks that repeatedly test an atomic condition in a tight loop. While spinlocks are suitable for extremely short critical sections, they are entirely unsuitable when a process must wait for slow events (such as disk I/O, user input, or cooperative process signaling) because spinning wastes valuable CPU cycles and cannot tolerate preemption.

- **Sleep Locks and Semaphores:** Blocking synchronization primitives that transition the calling process from the `RUNNING` state into a `SLEEPING` state, relinquishing the physical CPU core to other runnable processes until an explicit event wakes it.

In the MIT xv6 teaching operating system, the kernel provides a sleep-and-wakeup mechanism centered around wait channels. To prevent the ”lost wake-up problem” where an event signals between a condition check and the transition into sleep, xv6 pairs wait channels with spinlocks and explicit preparation functions (`sleep_prepare` and `sleep`).

1.2 Laboratory Objectives

The core goals of this laboratory work are:

a) Implement counting semaphores inside the xv6-riscv kernel by introducing three new system calls: `sem_init`, `sem_down`, and `sem_up`.

b) Maintain a shared kernel-level semaphore table protected by spinlocks and utilizing xv6 wait-channel primitives.

c) Wire the new system calls through all kernel layers: system call table, userland stubs, and process trap dispatchers.

d) Verify correct operation and ordering using a dedicated userland test utility (`user/semtest.c`).

e) Observe and resolve multi-core console output interleaving using a shared kernel semaphore in `user/mix.c`.

f) Provide rigorous answers to all theoretical and architectural observation questions regarding race conditions, lock releases during sleep, and loop condition re-evaluation.

2 PART 1: KERNEL SEMAPHORE ARCHITECTURE

2.1 System Architecture and Data Structures

A dedicated kernel source file (`kernel/sem.c`) was integrated into the xv6 kernel build. A static table of 8 semaphores was allocated in kernel memory:

```
#define NSEM 8

struct sem {
  struct spinlock lock;
  int value;
};

static struct sem sems[NSEM];
```

Each entry combines an integer counter (`value`) with an xv6 `spinlock` (`lock`) that guarantees atomic manipulation of the semaphore state across multiple CPU cores.

2.2 System Call Registration

The three system calls were registered with the operating system as follows:

- `SYS_sem_init` assigned ID **24** in `kernel/syscall.h`.
- `SYS_sem_down` assigned ID **25** in `kernel/syscall.h`.
- `SYS_sem_up` assigned ID **26** in `kernel/syscall.h`.

In `kernel/syscall.c`, external prototypes and handler dispatch pointers were established:

```
extern uint64 sys_sem_init(void);
extern uint64 sys_sem_down(void);
extern uint64 sys_sem_up(void);

static uint64 (*syscalls[])(void) = {
...
[SYS_sem_init] = sys_sem_init,
[SYS_sem_down] = sys_sem_down,
[SYS_sem_up]   = sys_sem_up,
};
```

Userland prototypes were added to `user/user.h`:

```
int sem_init(int id, int value);
int sem_down(int id);
int sem_up(int id);
```

Assembly trap stubs were added to `user/usys.pl`:

```
entry("sem_init");
entry("sem_down");
entry("sem_up");
```

Finally, `kernel/sem.o` was added to the kernel objects list `OBJS` in the xv6 `Makefile`, and user programs `_semtest` and `_mix` were appended to `UPROGS`.

2.3 System Call Implementations (`kernel/sem.c`)

The complete kernel logic is implemented as follows:

```
// sem.c: counting semaphores inside the xv6 kernel (Lab 13).
#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;
  int value;
};

static struct sem sems[NSEM];

// sem_init(id, value): assign semaphore start value.
uint64
sys_sem_init(void)
{
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;

  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

// sem_down(id): decrement value; sleep if 0.
uint64
sys_sem_down(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;

  struct sem *s = &sems[id];
  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);
    release(&s->lock);
    sleep();
    acquire(&s->lock);

    if (killed(myproc())) {
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): increment value and wake sleeping processes.
```

```
62 uint64
63 sys_sem_up(void)
64 {
65   int id;
66   argint(0, &id);
67   if (id < 0 || id >= NSEM)
68     return -1;
69
70   struct sem *s = &sems[id];
71   acquire(&s->lock);
72   s->value++;
73   wakeup(s);
74   release(&s->lock);
75   return 0;
76 }
```

3 PART 2: TEST SUITE VERIFICATION (`semtest.c`)

3.1 Test Program Logic

To validate kernel semaphore reliability, a user-space testing harness (`user/semtest.c`) was constructed covering two critical behavioral scenarios:

a) **Order Synchronization (Test 1):** Semaphore 0 is initialized to 0. A child process is forked and immediately calls `sem_down(0)`. The parent process sleeps for 10 ticks via `pause(10)`, ensuring the child reaches the sleep queue first. The parent then prints its message and calls `sem_up(0)`. The child is awakened and completes.

b) **Lockstep Ping-Pong (Test 2):** Two semaphores are coordinated: Semaphore 1 initialized to 1 (ping turn) and Semaphore 2 initialized to 0 (pong turn). Parent and child execute a synchronized ping-pong dialogue over 3 alternating iterations.

```
// semtest.c: two tests for the kernel semaphores (Lab 13).
#include "kernel/types.h"
#include "user/user.h"

int main(void) {
  // Test 1: order. Semaphore 0 starts at 0.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

3.2 Execution Output

The operating system was built and executed in QEMU:

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
pong 1
ping 2
pong 2
ping 3
pong 3
semtest: OK
```

Figure 3.1: Figure 3.1 – Verification trace of semtest confirming correct ordering and ping-pong execution

The output confirms that the child never preempts the parent during Test 1, and alternating execution proceeds without deadlocks or missed wakeups, concluding with `semtest: OK`.

4 PART 3: MULTIPROCESSING INTERLEAVING WITH `mix.c`

4.1 Console Contention Analysis

In `user/mix.c`, two processes print 5 lines of 30 characters each. Each character is written individually via a separate system call: `write(1, p, 1)`.

```
// mix.c: two processes print five lines each, one letter per write().
#include "kernel/types.h"
#include "user/user.h"

void lines(char *s, int uselock) {
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int main(int argc, char *argv[]) {
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

4.2 Execution Without Synchronization (`mix`)

When executed without arguments (`uselock = 0`), both processes access the console UART concurrently:

```
$ mix
AAbAAAbAbbbAbAbbAbbAbbAbAAAbAb
AbAbAbbAbAbbAbAbAbAbAbAbAbAbAb
AbAbAbAbAbAbAbAbAbAbAbAbAbAbAb
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

As shown above, individual characters 'A' and 'b' interleave randomly across columns, creating jumbled lines.

4.3 Execution With Semaphore Locking (`mix lock`)

When executed with the argument `lock` (`uselock = 1`), Semaphore 3 (initialized to 1) protects the entire 30-character line printing loop:

```
$ mix lock
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
```

Figure 4.1: Figure 4.1 – Strict line-by-line atomicity achieved under mix lock

With the semaphore acting as a mutex around each line, each 30-character sequence prints atomically without interruption.

5 OBSERVATION QUESTIONS AND ARCHITECTURAL ANALYSIS

a) **In the first run (without `lock`), why do the letters mix?**

In `mix.c`, character output is not written as a single buffered string; rather, each individual character is emitted via its own distinct system call: `write(1, p, 1)`.

Under xv6 on QEMU, the virtual machine is configured with multiple virtual CPUs (e.g., `-smp 3`). The parent process and child process run concurrently on separate physical CPU cores. Each invocation of `write()` crosses the user-kernel privilege boundary into the console driver independently. Because no synchronization protects the loop of 30 writes, system call traps from both cores interleave arbitrarily at the hardware UART controller, causing characters from both processes to blend together.

b) **Which two lines in `mix.c` make one line a critical section?**

The critical section is bounded by:

- **Line 10:** `sem_down(3);` immediately preceding the loop of character writes.
- **Line 14:** `sem_up(3);` immediately following the emission of the terminating newline character.

By acquiring Semaphore 3 before writing the first character of a line and releasing it only after writing the newline, the entire sequence of 31 single-byte `write()` system calls executes as an isolated atomic critical section.

c) **In `sem_down`, why `while (s->value == 0)` and not `if`?**

When a thread or process calls `wakeup(s)` inside `sys_sem_up`, xv6 marks **all** processes sleeping on the wait channel `s` as `RUNNABLE`.

When multiple sleeping processes wake up, they cannot proceed simultaneously because each process must first acquire `s->lock`. The first process to win the spinlock acquires it, decrements `s->value` from 1 to 0, and releases the spinlock. When subsequent awakened processes acquire the spinlock in turn, `s->value` is already 0.

If `if` were used, the subsequent processes would proceed unconditionally, decrementing `s->value` to -1, violating the fundamental non-negative semaphore invariant. The `while` loop forces every awakened process to re-evaluate whether a resource unit is genuinely available; if not, it safely re-enters `sleep()`.

d) **In `sem_down`, why `release(&s->lock)` before `sleep()`?**

If a process were to enter `sleep()` while still holding `s->lock`, an immediate and unrecoverable **kernel deadlock** would occur:

1) The process enters sleep holding the spinlock `s->lock`.

2) Another process attempts to execute `sem_up` to increment the semaphore and wake the sleeping process.

3) In `sys_sem_up`, the second process executes `acquire(&s->lock)`.

4) Because the sleeping process holds `s->lock`, the second process spins forever waiting for the lock.

5) Because the second process is locked spinning, it can never reach `wakeup(s)`.

6) Because `wakeup(s)` is never called, the sleeping process never wakes up to release `s->lock`.

By registering the process on the wait channel with `sleep_prepare(s)` and releasing `s->lock` before relinquishing the CPU in `sleep()`, xv6 guarantees that other processors can acquire `s->lock` and execute `wakeup(s)` safely without deadlocks or missed wakeups.

6 CONCLUSIONS

In this laboratory work, counting semaphores were successfully engineered and integrated directly into the xv6 operating system kernel.

The implementation spanned multiple architectural layers of the kernel: defining a centralized table of spinlock-protected semaphore descriptors, implementing atomic state transitions using xv6 sleep-and-wakeup channels, establishing trapframe argument unmarshaling via `argint()`, and exposing syscall stubs to userland processes.

Verification via `semtest` demonstrated exact ordering enforcement and deterministic two-process ping-pong synchronization. The multiprocessing console analysis in `mix` demonstrated how multi-core instruction interleaving corrupts unprotected shared device streams, and proved that kernel-level binary semaphores effectively restore critical section atomicity across concurrent processes.

Finally, deep architectural analysis clarified the critical importance of predicate re-checking with `while` loops upon wakeup and verified why holding spinlocks across sleep boundaries results in catastrophic system deadlocks.

REFERENCES

[1] Russ Cox, Frans Kaashoek, and Robert Morris, *xv6: a simple, Unix-like teaching operating system*, MIT CSAIL, 2024.

[2] Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson, 4th Edition, 2014.

[3] David A. Patterson and John L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, Morgan Kaufmann, 2nd Edition, 2020.

[4] Department of Software Engineering and Automation, *Operating Systems Lab 13: Semaphores in Your Own Kernel*, FCIM, Technical University of Moldova, 2026.